

SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-004: Student Input Field Validation and Strong Password Enforcement

Change Request ID	CR-004
Change Request Title	Student Input Field Validation and Strong Password Enforcement
Student Name	AAM. Nusky
Student ID	JF/ICT/24/01
Implementation Status	Completed
Date	8 October 2026

1. Change Request Summary

1.1 Business Requirement

CR-004 requires comprehensive validation for student Add and Edit operations. The system must prevent incomplete, incorrectly formatted, or insecure student information from being submitted and stored. Validation is required on both the frontend and backend so that invalid data cannot be accepted by bypassing the user interface.

1.2 Existing Behaviour

- StudentFormModal.jsx performed basic required-field checking only.
- Phone numbers with fewer or more than 10 digits could potentially be accepted.
- Non-numeric characters could be entered into the phone number field.
- First and last names did not enforce a minimum 3-character requirement.
- Name fields did not comprehensively reject invalid characters.
- Email validation was limited to the browser input type.
- Registration number validation was not comprehensive.
- New student passwords did not enforce the required strong password policy.
- Backend validation did not comprehensively validate all student input fields.

1.3 Requested Change

- Validate all required student fields before submission.
- Require a valid registration number.
- Require first and last names to contain at least 3 characters and valid alphabetic characters.
- Validate email addresses using a valid email format.
- Require phone numbers to contain exactly 10 numeric digits.
- Require strong passwords for new students: minimum 8 characters, uppercase, lowercase, number, and special character.
- Display clear validation errors.
- Prevent invalid frontend submissions.
- Apply equivalent server-side validation to prevent client-side validation bypass.
- Preserve existing Add, Edit, Delete, authentication, authorization, and password hashing functionality.

2. Implementation

2.1 Implementation Summary

CR-004 was implemented by enhancing the administrator student Add/Edit form and the existing student controller validation. Frontend validation was added to StudentFormModal.jsx with field-specific validation errors. Server-side validation was added to studentController.js so that direct API requests containing invalid data are rejected before database operations are performed.

The existing student model and database operations were reviewed. No database schema changes were required, and studentModel.js remains unchanged. Existing bcrypt password hashing, parameterized database queries, authentication, authorization, and CRUD operations were preserved.

2.2 Affected Files

File	Change	Purpose
frontend/src/components/StudentFormModal.jsx	Modified	Implemented client-side validation, field-specific error messages, phone input restrictions, and prevention of invalid Add/Edit submission.
backend/controllers/studentController.js	Modified	Implemented server-side validation for registration number, names, email, phone number, and new-student passwords; validation is applied to Add and Edit operations.
backend/models/studentModel.js	Reviewed / No change	Existing parameterized CRUD queries and student data access were retained; no schema-related validation change was required.

Additional files: None.

2.3 Technical Changes

Frontend Validation

- Added a validateForm() function to validate all student fields before an API request is sent.
- Added fieldErrors state for displaying field-specific validation messages.
- Registration number is required and restricted to letters, numbers, slash, and hyphen.
- First name and last name require at least 3 characters and valid alphabetic name characters.
- Email address is checked against a valid email pattern.
- Phone number is restricted to numeric input and exactly 10 digits; maxLength is set to 10.
- Password validation is applied when creating a new student.
- Invalid data prevents the submit handler from sending the API request.
- Existing authentication error and authorization error handling remains unchanged.

Backend Validation

- Added reusable validation helper functions for registration number, names, email, phone number, password, and complete student data.
- Validation is performed after trimming incoming string values.
- Invalid requests return HTTP 400 with a general validation message and field-specific error details.
- Create operation validates the password before database insertion.
- Edit operation validates all editable student fields before update.
- Duplicate registration number and duplicate email checks remain enforced.

- Existing transaction handling for student creation remains unchanged.
- Passwords continue to be hashed using bcrypt before storage.
- Database queries remain parameterized.

2.4 Validation Rules Implemented

Field	Validation Rule	Operations
Registration Number	Required; minimum 2 characters; letters, numbers, / and - only.	Add / Edit
First Name	Required; minimum 3 characters; alphabetic name characters only.	Add / Edit
Last Name	Required; minimum 3 characters; alphabetic name characters only.	Add / Edit
Email	Required; valid email format.	Add / Edit
Phone Number	Required; numeric characters only; exactly 10 digits.	Add / Edit
Password	Required for new student; minimum 8 characters; uppercase, lowercase, number, special character.	Add only

2.5 Strong Password Validation

For new student creation, the backend and frontend enforce the following password policy:

- Minimum 8 characters.
- At least one uppercase letter.
- At least one lowercase letter.
- At least one number.
- At least one special character.

The existing bcrypt hashing mechanism remains unchanged. Passwords are not returned in the success response and are not stored as plain text.

2.6 Implementation Logic

1. Administrator opens Add Student or Edit Student.
2. The form loads the appropriate student data.
3. Administrator enters or edits student information.
4. Frontend validation checks the submitted values.
5. If validation fails, field-specific errors are displayed and the API request is prevented.
6. If frontend validation succeeds, the existing Add or Edit API request is sent.
7. Backend validation repeats the critical validation rules.
8. Invalid direct API requests are rejected with HTTP 400 before database modification.
9. For a valid new student, the password is hashed using bcrypt and the existing transaction creates the account and profile.
10. For a valid edit, the existing student record is updated without changing the existing CRUD flow.

3. Testing & Verification

3.1 Testing Approach

The implementation was prepared for manual frontend and API verification. The following test cases cover the CR-004 validation rules, Add/Edit operations, direct API validation, password security, and regression of existing CRUD functionality.

Environment: Development

Browser: [Chrome / Edge / Firefox]

Test Role: Administrator

Note: The implementation code was completed, but execution results/screenshots were not provided in the current evidence. Therefore, Actual Result and Status should be updated after running the tests.

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Submit with empty required fields	Submission is blocked and required-field errors are displayed.	To be verified	Pending
TC-02	First name with fewer than 3 characters	First name is rejected.	To be verified	Pending
TC-03	Last name with fewer than 3 characters	Last name is rejected.	To be verified	Pending
TC-04	Names containing numbers/special invalid characters	Invalid names are rejected.	To be verified	Pending
TC-05	Invalid email format	Email is rejected.	To be verified	Pending
TC-06	Phone number with fewer than 10 digits	Phone number is rejected.	To be verified	Pending
TC-07	Phone number with more than 10 digits	Phone number is rejected.	To be verified	Pending
TC-08	Phone number containing letters	Phone number is rejected.	To be verified	Pending
TC-09	Valid 10-digit phone number	Phone number is accepted.	To be verified	Pending
TC-10	Password shorter than 8 characters	Password is rejected.	To be verified	Pending
TC-11	Password without uppercase/lowercase/number/special character	Weak password is rejected.	To be verified	Pending
TC-12	Valid strong password	Password is accepted.	To be verified	Pending
TC-13	Create student with valid information	Student is successfully created.	To be verified	Pending
TC-14	Create student with invalid information	Student is not created.	To be verified	Pending
TC-15	Update student with invalid information	Update is rejected.	To be verified	Pending
TC-16	Update student with valid information	Student is successfully updated.	To be verified	Pending

TC-17	Direct API request with invalid data	Backend returns HTTP 400 and no invalid data is stored.	To be verified	Pending
TC-18	Delete existing student after validation changes	Correct student is deleted and existing Delete functionality remains operational.	To be verified	Pending
TC-19	Administrator authentication/authorization regression	Existing access controls continue to work.	To be verified	Pending
TC-20	Verify password storage	Password is stored using the existing bcrypt hash mechanism, not plain text.	To be verified	Pending

3.3 Test Summary

Total Test Cases	20
Passed	Pending verification
Failed	Pending verification
Pass Rate	To be calculated after execution
Overall Test Result	Pending

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification	Status
AC-001	Required fields cannot be submitted empty.	Frontend validation implemented.	Implemented / Test Pending
AC-002	First names under 3 characters are rejected.	First-name validation implemented.	Implemented / Test Pending
AC-003	Last names under 3 characters are rejected.	Last-name validation implemented.	Implemented / Test Pending
AC-004	Invalid numbers/inappropriate characters in names are rejected.	Name regular-expression validation implemented.	Implemented / Test Pending
AC-005	Invalid email addresses are rejected.	Frontend and backend email validation implemented.	Implemented / Test Pending
AC-006	Phone numbers under 10 digits are rejected.	Exact 10-digit validation implemented.	Implemented / Test Pending
AC-007	Phone numbers over 10 digits are rejected.	Frontend max length and backend exact-length validation implemented.	Implemented / Test Pending
AC-008	Non-numeric phone numbers are rejected.	Numeric-only validation implemented.	Implemented / Test Pending
AC-009	Valid 10-digit phone number is accepted.	Validation rule implemented.	Implemented / Test Pending
AC-010	Weak passwords are rejected for new students.	Strong password validation implemented.	Implemented / Test Pending
AC-011	Strong password with all required character types is accepted.	Password policy implemented.	Implemented / Test Pending

AC-012	Valid student can be created.	Existing create flow preserved with validation.	Implemented / Test Pending
AC-013	Invalid student cannot be created.	Frontend and backend validation implemented.	Implemented / Test Pending
AC-014	Invalid student cannot be updated.	Backend Edit validation implemented.	Implemented / Test Pending
AC-015	Valid student can be updated.	Existing update flow preserved.	Implemented / Test Pending
AC-016	Direct API invalid data is rejected.	Server-side validation returns HTTP 400.	Implemented / Test Pending
AC-017	Existing Edit and Delete affect the correct student.	Existing student_id-based CRUD preserved.	Implemented / Test Pending
AC-018	Administrator authentication and authorization continue to work.	Existing auth/authz code was not changed.	Implemented / Test Pending

5. Evidence

The final submission should attach a maximum of two relevant screenshots, following the sample implementation report format.

Evidence 01 – Student Add/Edit Validation

Suggested screenshot: Student Add Student form showing one or more validation errors, such as invalid name, invalid email, invalid phone number, or weak password.

Related: AC-001 to AC-011

Evidence 02 – Backend Rejection / Valid CRUD

Suggested screenshot: Postman showing an invalid student request rejected with HTTP 400, or a valid student creation/update together with the resulting successful response.

Related: AC-012 to AC-018

The screenshot displays the 'Student Management System' Administrator Dashboard. A modal window titled 'Add New Student' is open, showing a form to create a new student account. The form contains the following fields and validation messages:

- Registration Number:** REG003
- First Name:** a. Validation message: "First name must contain at least 3 characters."
- Last Name:** b. Validation message: "Last name must contain at least 3 characters."
- Email:** admin123@gmail.com
- Phone Number:** 0652221234
- Password:** (masked with dots)

A red error message at the top of the form states: "Please correct the validation errors before submitting." The form has 'Cancel' and 'Create Student' buttons at the bottom. In the background, the dashboard shows a list of students with columns for 'REGISTRATION NO.', 'NAME', and 'ACTIONS'.

6. Final Result

Requirements	Implemented; testing pending
Implementation	Completed
Testing	Pending execution / evidence
Acceptance Criteria	Implementation completed; verification pending
Existing CRUD	Code preserved; regression test pending
Final Status	Pending final test verification

6.1 Final Summary

CR-004 was implemented as a frontend and backend validation enhancement for the administrator student Add and Edit operations. StudentFormModal.jsx now validates registration number, first name, last name, email, phone number, and new-student password before submitting an API request. Field-specific validation messages are displayed and invalid frontend submissions are blocked.

Server-side validation was added to studentController.js to prevent client-side validation bypass. The backend validates the same critical student fields and rejects invalid requests before database modification. New student passwords must meet the strong password policy and continue to be securely hashed using bcrypt.

The existing student model, parameterized database queries, authentication, authorization, Add/Edit/Delete functionality, and database schema were preserved. No new API endpoint or database migration was required.

6.2 Issues / Limitations

- Formal manual frontend and Postman/API test execution is still required to populate actual results.
- Validation screenshots/evidence must be captured after running the test cases.
- No database schema redesign was performed, in accordance with the CR-004 scope.
- Registration number validation allows letters, numbers, slash, and hyphen; the exact institutional registration-number format was not specified in the change request.

Student Name: AAM. Nusky

Student ID: JF/ICT/24/01

Date: 8 October 2026

Signature: _____

~ END OF REPORT ~